

AI Developers Course — Final Project

HR Copilot

An AI-based Personal Assistant for Recruiters — System Specification

Author: Miguel Rosenberg

Scenario: #3 — AI-based Personal Assistant for Recruiters

Build: HRCopilot_Project_v2, `n8n-orchestrated` branch

Date: September 2026

LangGraph · 6 nodes

RAG · Qdrant Cloud

n8n · 27-node workflow

Flask backend

Docker Compose

3 guardrails

Contents

1. Problem & Solution

2. Node Detail (LangGraph)

3. RAG Strategy

4. n8n Automation & Flask Backend

5. Tool Descriptions

6. Runtime Environment (Docker)

7. Verification & Test Scenarios

8. Deliverable Structure & How to Run

9. Bonus (Optional) — Statistical Fit Model

1 Problem & Solution Description

The problem

Screening candidates against a job requisition is repetitive, manual work: a recruiter reads a stack of resumes, compares each one against the role's requirements, decides who is worth an interview, then manually finds a free calendar slot, writes an invitation email, and logs the outcome in the ATS (Applicant Tracking System). None of that is intellectually hard — it is slow, inconsistent between recruiters, and easy to get wrong under volume (a strong candidate skimmed too quickly, an interview double-booked, a candidate never logged).

The solution

HR Copilot takes a free-text job requisition and automates the entire screening-to-scheduling pipeline:

1. Parses the requisition into structured requirements (title, required skills, minimum experience, headcount).
2. Retrieves the most relevant candidate CVs from a vector store (RAG), combining semantic similarity with a hard metadata filter on years of experience.
3. Autonomously scores every retrieved candidate against the role, with a written justification grounded in the actual CV text — not a fixed rule, but an LLM reasoning over each CV individually, cross-checked against a statistical model.
4. For every candidate who clears the bar, automatically books an interview slot, sends an invitation, and logs the candidate to the ATS — a real, persisted change in an external system, not just a displayed answer.
5. Returns a screening report the recruiter can read in seconds, listing who was evaluated, why, who passed, and what was booked.

The value

What used to take a recruiter tens of minutes per requisition — reading CVs, deciding, scheduling, writing an email, updating a spreadsheet — happens in roughly a minute, with two independent safety nets: the system will not act on off-topic or adversarial input, and it will not let the model's account of a candidate outrun what the CV actually supports. When nobody is a fit, the system says so honestly instead of forcing a match.

Why this is more than a chatbot. The brief explicitly prohibits a static Q&A system. HR Copilot's core action — deciding whom to interview and then actually reserving a slot, sending mail, and writing an ATS row — is a state change in the outside world, driven by an autonomous, non-deterministic agent (Node 3) rather than a fixed `if/else` rule.

Architecture at a glance

n8n is the orchestrator: one webhook drives the whole run. It screens the input, calls the LangGraph agent once over HTTP, screens what comes back, and only then performs the calendar/mail/ATS actions. LangGraph screens; it does not act. Every route to a real side effect passes through both n8n guardrails — there is no other door to Gmail, Google Calendar, or the ATS log.

2 Node Detail (LangGraph)

The graph is built with `langgraph.graph.StateGraph` over a shared `HRCopilotState` (a `TypedDict`). It has **six** nodes (the brief requires at least five) and **two conditional edges** (at least one required). n8n calls this graph exactly once per run, over HTTP, between its own input guardrail and its own shortlist guardrail — the graph itself makes no outbound network calls.

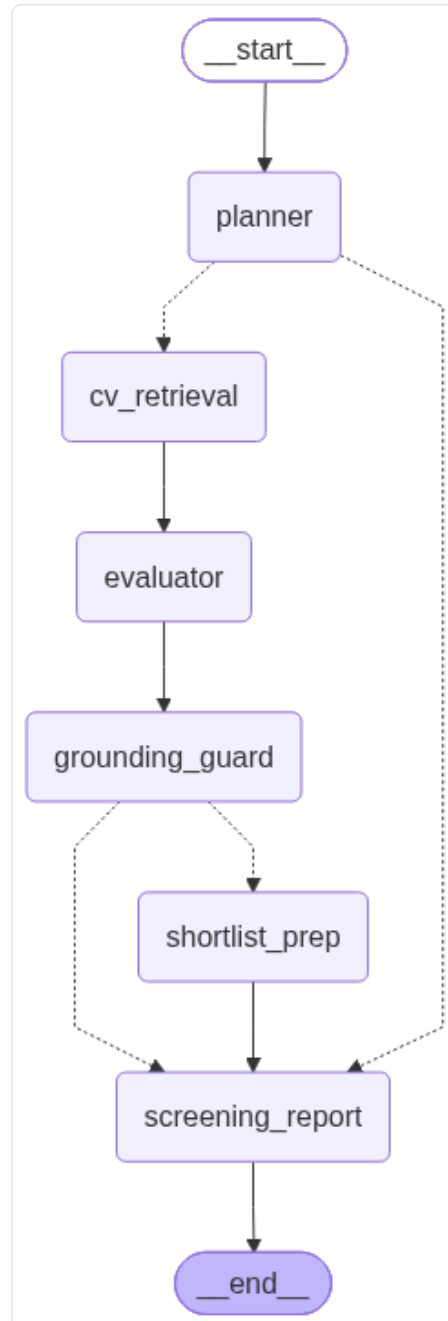


Figure 2 — Compiled graph, generated via `graph.get_graph().draw_mermaid_png()` (mandatory deliverable, also at `docs/langgraph_visualization.png`).

1 Planner

Turns the free-text requisition into a structured `JobRequirements` object (`title` , `required_skills` , `min_years_experience` , `headcount`) using a structured-output LLM call (`create_agent(..., response_format=JobRequirements)`). An empty requisition is rejected deterministically before any LLM call. A requisition that fails to parse stops the run with a reason, rather than searching on empty criteria.

Note: the *safety* input guardrail (prompt-injection / off-topic / jailbreak) does not live here — it runs earlier, in n8n, before this graph is even invoked (see Section 4). By the time Node 1 runs, the text has already been accepted as a genuine job requisition; Node 1's own check is a parsing concern, not a safety one.

2 CV Retrieval (RAG)

Embeds a query built from the parsed title and required skills, searches the Qdrant collection `hr_copilot_resumes` , and combines the semantic similarity search with a **metadata filter** on `years_experience ≥ min_years_experience` — the "advanced retrieval" the brief recommends beyond plain similarity search. If the filter excludes every candidate, the search retries without it, so the Evaluator can still judge (and honestly reject) rather than the recruiter seeing an empty result with no explanation. Results are deduplicated by candidate name, since a CV can be split across several chunks.

Because the filter is a hard cut, it can also remove the single best match for a role over a one-year shortfall while less relevant people who clear the bar take their place. The node therefore re-runs the same search unfiltered and records any excluded candidate who **outranked every candidate that was actually evaluated**, for Node 6 to report. Outranking the whole evaluated set is a deliberately strict test: with a small CV pool an unfiltered search returns most of the store, so a looser rule would list an unrelated candidate at rank 8 and imply they were worth considering. These candidates are named, never scored — the Evaluator never sees them.

3 Evaluator (autonomous)

The core autonomous decision in the graph. For each retrieved candidate, a separate `create_agent` instance (structured output `CandidateScore`) compares the CV against the role and produces a 0–10 fit score, a justification grounded in the CV, and matched/missing skill lists. The model is given **one tool**, `statistical_fit_score` (Section 5 and Section 9), which it must call once and then reason past — agreeing with the tool's estimate by default is explicitly treated as a failure in the system prompt. Candidates are scored in parallel on a thread pool (independent LLM calls; retrieval can return up to 8 candidates, and sequential scoring made this the slowest part of the graph by a wide margin). Candidates scoring $\geq$ `SCORE_THRESHOLD` (7.0 by default) become `passing_candidates` .

Not fully deterministic, by design: the same CV can score differently across runs because the LLM's reasoning — not a fixed rule — decides the outcome. This is the system's "several reasoning iterations until it reaches an answer" requirement: score → tool call → reconsider → final structured answer.

4 Grounding Guard (local output guardrail)

OUTPUT GUARDRAIL #1

A **deterministic** check (no LLM): for every candidate, does each skill the Evaluator listed under `matched_skills` actually appear (case-insensitive substring) in that candidate's CV text? If not, the candidate's justification is rewritten in place with a safe, fact-only version that withholds the unverified skill, before *anything* downstream can use it. This node runs **before** Node 5 builds the ATS/email payload — earlier project iterations ran this check last, which meant an unverified claim could already have reached the ATS row and the invitation email by the time it was caught. An earlier version of this check asked the LLM to self-verify and it produced self-contradictory flags; a plain substring test proved more reliable than the model at its own stated task.

5 Shortlist Prep

No LLM call, no network call. Assembles the two artefacts n8n needs: `actions_request` (role, mode, and per-candidate name/email/delivery address/score/notes for the calendar-mail-ATS chain) and `shortlist_text` (role, required skills, and each passing candidate's current job title only — deliberately no CV text or justification, since the only thing n8n's shortlist guardrail needs to judge is job-family relevance). Reached only via **Conditional Edge 2** (below); when nobody passed, this node is skipped entirely and the empty shortlist is itself the stop.

6 Screening Report

Renders the recruiter-facing text: every candidate evaluated and their score, who passed, and — when the LLM score and the statistical model disagree by 2.5 points or more — an explicit "worth a second look" flag naming the gap. It also names any candidate the experience filter held back from evaluation despite outranking everyone who was evaluated (Node 2), so a shortlist emptied by the minimum-years bar rather than by the candidate pool says so instead of presenting a page of poor fits with no explanation. Also the single place that reports a parse failure or a retrieval-layer error honestly, rather than presenting either as "no qualifying candidates" (a real bug this distinction was added to catch). This node says nothing about interviews or emails: n8n appends its own "Actions taken" block to this text after it has actually performed them, so the report never claims a side effect before it has happened.

Conditional edges

Edge	Condition	Routes to
Conditional Edge 1 <code>route_after_planner</code>	Did the Planner produce a usable <code>JobRequirements</code> ?	parsed → <code>cv_retrieval</code> not parsed → <code>screening_report</code> (skips retrieval and scoring entirely)
Conditional Edge 2 <code>route_after_grounding</code>	Did at least one candidate clear <code>SCORE_THRESHOLD</code> ?	yes → <code>shortlist_prep</code> (builds the n8n action request) no → <code>screening_report</code> directly (nothing to act on)

Both edges branch on the *content* of the state (a real LLM outcome), not a fixed rule evaluated before any reasoning happens — satisfying the brief's "more than a simple if X then Y" requirement.

3 RAG Strategy

Source data

Ten synthetic candidate resumes (PDF), spanning eight professions (backend engineer, data analyst, DevOps/SRE, UX designer, sales/account executive, etc.), generated with AI per the course's guidance (spec §7) on producing realistic mock data for RAG testing. Each resume follows a fixed layout — name, role title, a `Skills:` line, a `Years of experience:` line, and an `Email:` line — so structured metadata is parsed deterministically from the extracted text rather than via an extra LLM call: cheaper, reproducible, and easy to explain.

Chunking strategy

Splitter: `RecursiveCharacterTextSplitter`
Chunk size: 1200 characters · **Overlap:** 200 characters

Resumes are short, self-contained documents, so most stay within one or two chunks; splitting mainly guards against an unusually long CV rather than doing any real semantic segmentation.

The important design choice: every chunk carries the *same* full metadata — candidate name, role title, years of experience, skills, email — *and* the complete resume text (`full_text`), not just its own fragment.

This means the Evaluator (Node 3) always scores a **whole person**, never a fragment that happened to match the query well but is missing context that would change the verdict.

Embedding model

`text-embedding-3-small` (OpenAI), 1536 dimensions, cosine distance. Chosen for low latency/cost on short documents where the marginal retrieval quality of a larger embedding model is not worth the extra spend — ten candidates and a handful of requisitions is not a workload that benefits from a bigger model.

Vector database

Qdrant Cloud (a hosted cluster, not a bundled container — reached via `QDRANT_URL` / `QDRANT_API_KEY`), collection `hr_copilot_resumes` . A payload index is created on `metadata.years_experience` at ingest time; Qdrant Cloud rejects a range filter on an unindexed field, so without this index every requisition asking for N+ years would silently return zero candidates.

Advanced retrieval (beyond plain similarity search)

Technique	How it is used
Metadata filtering	A Qdrant <code>FieldCondition</code> / <code>Range</code> filter on <code>years_experience >= min_years_experience</code> is applied at query time, so a requisition asking for 5+ years never even considers a 2-year candidate whose CV happens to use similar wording.
Graceful filter fallback	If the experience filter excludes everyone, the same query re-runs without it. The Evaluator still sees and can honestly reject the shortfall, instead of the recruiter getting an empty result with no visible reason.
Query construction	The embedded query text is <code>"{title}. Required skills: {skill list}"</code> , built from the <i>parsed</i> requirements rather than the raw requisition — this matches how the CVs themselves are phrased and retrieves noticeably better than embedding the requisition's full boilerplate text.

Deduplication

Hits are deduplicated by candidate name (best-scoring chunk first), since a long CV can be split across multiple chunks and the Evaluator must score one whole person, not re-score the same person twice under two chunks.

4 Automation Flow (n8n) & Backend Server (Flask)

Why n8n orchestrates and LangGraph only screens

There is exactly **one** webhook, `POST /webhook/screen`. It runs an input guardrail, calls the LangGraph agent once over HTTP, runs a second guardrail on what comes back, and only then performs the calendar/mail/ATS actions. This ordering is deliberate: it guarantees that **every** path to a real side effect (Gmail, Google Calendar, the ATS log) passes through both guardrails — there is no separate "actions" webhook left unguarded.

Step-by-step n8n flow

Step	What happens
1. Screen Webhook	Receives <code>{requisition_text, session_id}</code> as JSON via <code>POST /webhook/screen</code> .
2. Guardrails (input)	<code>@n8n/n8n-nodes-langchain.guardrails</code> node running three checks: <code>keywords</code> , <code>jailbreak</code> , <code>topicalAlignment</code> (confidence threshold 0.7). Any check triggering routes to Explain Block → Respond (Blocked) with a rejection message — the LangGraph agent is never invoked, so no LLM scoring cost is paid on a rejected request.
3. Screen Agent	HTTP POST to <code>http://agent:5000/screen</code> with the requisition text. This is the entire two-way integration: n8n calls into the Flask/LangGraph service and receives back structured JSON (requirements, scored candidates, <code>shortlist_text</code> , <code>actions_request</code> , <code>report</code>).
4. Shortlist Check	A second Guardrails node, this time with a <code>custom</code> check: an LLM is asked whether the passing candidates plausibly belong to the requisitioned profession, reading only <code>shortlist_text</code> (role, required skills, job titles — no CV bodies). Configured to fail closed : if the check itself errors, the run is treated as blocked rather than passed, since a broken guardrail must never silently read as "all clear" once it gates real actions.
5. Has Passing?	A Switch/branch on whether the shortlist is non-empty. Empty → Compose Reply → Respond with the screening-only report; non-empty → the action chain.
6. Action chain	<code>Read Calendar</code> → <code>Assign Slots</code> → <code>Write Calendar</code> → <code>Read Outbox</code> → <code>Compose Invitations</code> → <code>Write Outbox</code> → <code>Read ATS Log</code> → <code>Append ATS Entries</code> → <code>Write ATS Log</code> → <code>Build Response</code> . Deterministic Code nodes operating on the <i>whole</i> shortlist in one pass — one read and one write per file per run, not per candidate, which is what avoids a read-before-write race across candidates.
7. Respond to Webhook	Returns the screening report plus an <code>Actions taken:</code> block (who was booked, who had no free slot, what was logged) as structured JSON.
8. (optional) Fan Out Live	Only when <code>ACTION_MODE=live</code> : fires <i>after</i> the webhook has already responded, sending a real Gmail invitation and creating a real Google Calendar event per booked candidate. The default, <code>ACTION_MODE=log</code> , only writes to the mounted JSON files — still a genuine, persisted external state change, and the mode the brief's requirement is satisfied by without needing any Google OAuth setup.

n8n workflow requirements checklist (spec §4.3): starts with a Webhook node receiving JSON ✓; includes logic beyond the webhook — a Guardrails node, Switch/branch logic, and file read/write operations ✓; ends with Respond to Webhook returning structured data the agent/caller can process ✓.

Flask backend

The Flask server (`src/app.py`) is the piece n8n calls into. It exposes:

Endpoint	Purpose
<code>POST /screen</code>	Body <code>{"requisition_text": str, "session_id": str?}</code> . Builds the compiled LangGraph once at import time, invokes it per request, and returns the full JSON result n8n reads. No side effects: it reads Qdrant and spends LLM calls, but books, sends and writes nothing — every route to a real action goes through n8n's guardrails, never directly through this endpoint.
<code>POST /candidates</code>	Multipart PDF upload; parses and embeds one new CV into the live Qdrant collection immediately (visible to the very next <code>/screen</code> call, no restart), and saves it to disk so a future <code>ingest.py --reset</code> picks it up too.
<code>GET /health</code>	Liveness plus config echo. <code>?deep=true</code> additionally checks Qdrant connectivity/point count and probes whether n8n's <code>screen</code> webhook is actually registered — n8n only registers webhooks when its own process starts, so a published workflow can silently stop answering while every container still reports healthy; this is the one signal that catches it.

The graph is compiled once at import time (wiring nodes/edges does not depend on the request), and the production entrypoint runs behind `gunicorn` with 2 worker processes and a 300s timeout matched to n8n's own HTTP Request node timeout.

5 Tool Descriptions

The brief asks for the formats/descriptions given to the autonomous agent so it knows when and how to use each tool. HR Copilot's autonomous reasoning is concentrated in Node 3, so that is where the one LLM-invoked tool lives; the "action tools" the original scenario describes (book a slot, send an email, log to the ATS) are deterministic once Node 3 has decided who passed, and now live in n8n as Code/HTTP nodes rather than as LLM-bound Python tools (see the design note at the end of this section).

statistical_fit_score — bound to the Evaluator (Node 3)

```
@tool
def statistical_fit_score(candidate_name: str) -> str:
    """Statistical fit estimate for the candidate you are currently reviewing.

    Produced by a small ridge-regression model fitted on hand-labelled (requisition,
    candidate) pairs. It sees only four coarse signals: the fraction of required skills
    matched by keyword, years above or below the minimum, job-title overlap, and
    vector-search similarity.

    It has NOT read the CV. It cannot tell a CV that mentions a technology once in passing
    from one with three projects using it, and it cannot see recency, depth or trajectory.
    It is one piece of evidence to weigh against the CV, and it is never the answer.

    Args:
        candidate_name: the candidate you are reviewing, exactly as written in the CV header.
    """
```

The tool is built as a **per-candidate closure** over that candidate's already-known data, rather than a function taking features as arguments. Tool arguments are filled in by the calling model, and an LLM cannot compute a real cosine similarity or keyword-overlap fraction — asking it to supply those numbers would mean asking it to invent them, and the moment the inputs are LLM-generated the statistical estimate stops being an independent second opinion. The single declared argument, `candidate_name`, exists only so the tool has a non-empty schema and a self-describing call in the message trace; it is not used to look anything up.

The system prompt explicitly instructs the model on *how* to use the return value: read it as a range with named drivers, never quote the point estimate as the model's own score, and treat agreeing with it by default as a failure rather than a success (see Section 9 for why).

Structured-output contracts (Nodes 1 and 3)

Rather than free-text tool calls, the Planner and Evaluator use LangChain's `response_format=` structured-output mode with pydantic models (`JobRequirements` , `CandidateScore`). These are not tools the model "chooses" to call, but they are the schema contract that keeps every autonomous node's output machine-readable by the rest of the graph — the model is forced to return exactly these fields, with types the rest of the pipeline can trust without re-parsing free text.

The n8n action layer (formerly LLM-bound tools)

Step	What it does
Assign Slots	Reads <code>calendar.json</code> , picks the next open slot per candidate, marks it booked.
Compose Invitations	Builds the invitation body from the slot just booked; a candidate with no free slot never gets an invitation promising a time — the rule is structural, not a prompt instruction to be trusted.

Append ATS Entries

Appends one row per candidate to `ats_log.json` with status, score, and the (already grounding-checked) notes.

Design note. An earlier iteration bound these three actions as LangChain tools directly to an LLM tool-calling loop in the agent (one `create_agent` loop per passing candidate: $3 \times N$ model turns and HTTP calls). They were moved to deterministic `n8n` nodes because none of that work is a judgement call — which slot is free, what an invitation says, and what goes in the ATS row are mechanical once Node 3 has decided *who* passes. The move also fixed a real concurrency bug: running the actions once per candidate meant N parallel read-modify-write cycles on the same JSON files, so one candidate's booking could silently overwrite another's. Handling the whole shortlist in a single pass reads and writes each file exactly once per run.

6 Runtime Environment (Docker)

Container structure

One `Dockerfile` (Python 3.12-slim base) is reused by three services via Compose command overrides, so there is a single dependency set and a single image to keep in sync:

Service	Port	Role
<code>n8n-import</code>	—	One-shot: provisions OpenAI/Gmail/Calendar credentials and imports + publishes the workflow <i>before</i> n8n's own process starts (n8n only registers webhooks at process boot, never on a live update — doing this pre-boot is what makes the webhook work from the very first request).
<code>n8n</code>	5679→5678	The orchestrator described in Section 4.
<code>ingest</code>	—	One-shot: chunks and embeds <code>data/resumes/*.pdf</code> into Qdrant Cloud (<code>--reset</code>).
<code>agent</code>	5001→5000	Flask + LangGraph, behind <code>gunicorn</code> (2 workers).
<code>ui</code>	8502→8501	Streamlit front end (bonus UI, spec §4.5).

External dependency

Qdrant runs as a **hosted Qdrant Cloud cluster**, not a local container, reached via `QDRANT_URL` / `QDRANT_API_KEY` . This is the one piece of the stack not brought up by `docker compose up` itself.

Required environment / dependencies

Mandatory	<code>OPENAI_API_KEY</code> , <code>QDRANT_URL</code> , <code>QDRANT_API_KEY</code>
Optional	<code>ACTION_MODE</code> (log/live, default log), <code>SCORE_THRESHOLD</code> (default 7), Gmail/Calendar OAuth vars for live mode
Ports (host)	UI 8502 · Agent 5001 · n8n 5679
Volumes	<code>n8n_data</code> (n8n's own DB) · <code>./n8n_data</code> host folder mounted at <code>/hr_data</code> holding <code>calendar.json</code> / <code>sent_emails.json</code> / <code>ats_log.json</code>

One command

```
cp .env.example .env
# edit .env: paste OPENAI_API_KEY, QDRANT_URL, QDRANT_API_KEY

docker compose up --build -d
```

That single command brings up the entire stack, embeds the ten seed CVs, and imports + publishes the n8n workflow before n8n's process ever starts — no manual n8n login, no manual credential setup, no manual webhook activation. First run: roughly 1–2 minutes (mostly the image build); subsequent runs are fast, since everything is cached in named volumes.

7 Verification & Test Scenarios

Per the brief's own recommended step ("run at least 5 usage scenarios: correct flow, error flow, edge cases"), the project ships a seven-case demo script ([docs/demo_script.md](#)) covering the system's full decision surface, plus a scripted health check.

#	Case	Reaches Evaluator?	Expected outcome
1	Senior Backend Engineer (clean match)	Yes	Alice Chen scores highest and clears threshold → interview booked, invitation sent, ATS logged. A near-miss (Java/Azure, not Python/AWS) is correctly held below threshold.
2	Data Analyst (multi-candidate)	Yes	Two plausible candidates retrieved and scored; demonstrates the statistical-model/LLM disagreement flag on a boundary case.
3	Off-topic ("What's the weather in Tel Aviv?")	No	Rejected by n8n's <code>topicalAlignment</code> check before the agent is ever called — zero LLM scoring cost.
4	Prompt injection ("ignore all previous instructions...")	No	Rejected by n8n's <code>jailbreak</code> check, same guarded path as case 3.
5	Legitimate but zero-match (ship captain role)	Yes	Every candidate scores low; report ends "No candidate reached the threshold" — accepted as valid input, rejected on merit.
6	Site Reliability Engineer (out-of-sample for the fit model)	Yes	Ryan Patel books; a role/requisition the statistical model has never seen, verifying the LLM's independent judgement still works.
7	Machine Learning Engineer (out-of-sample, harder rejection)	Yes	Nobody books, despite genuine skill overlap (Python, cloud) — correctly rejected on job family alone.

Health check (run before any demo)

```
curl "http://localhost:5001/health?deep=true"
```

Reports the Qdrant point count and whether n8n's `screen` webhook is actually registered. This check exists because n8n registers a workflow's webhook only when its own process starts — a published workflow can silently stop answering while `docker compose ps` still shows every container healthy, and this is the only signal that catches it before a demo.

Forcing each guardrail to fire (for a live demonstration)

- **Shortlist relevance guardrail:** temporarily set `SCORE_THRESHOLD=0` and resend case 5 (the ship captain) — every candidate "passes", the shortlist is obviously the wrong profession, and the n8n check blocks the run before `Read Calendar` ever executes. Verified by diffing `calendar.json` / `sent_emails.json` / `ats_log.json` before and after: byte-identical.
- **Grounding guard:** hand-construct a `CandidateScore` claiming a skill absent from a CV and call `check_grounding()` directly — it flags the unverifiable skill and the justification is replaced.

8 Deliverable Structure & How to Run

```
HRCopilot_Project_v2/
├── src/                                # Flask API, LangGraph nodes/state/graph, RAG ingestion, Streamlit UI
│   ├── app.py                         # /screen, /candidates, /health
│   ├── graph.py                       # StateGraph wiring (6 nodes, 2 conditional edges)
│   ├── nodes.py                       # the six node functions
│   ├── state.py                       # HRCopilotState + pydantic response schemas
│   ├── retrieval.py                   # RAG query + metadata filter (Node 2)
│   ├── ingest.py                      # chunk + embed data/resumes/*.pdf into Qdrant
│   ├── scoring_tool.py                # statistical_fit_score tool (Node 3)
│   ├── scoring_model.py               # pure-Python evaluation of fit_model.json
│   └── scoring_features.py             # the 5 model features
├── data/
│   ├── resumes/                       # 10 synthetic candidate CVs (PDF)
│   ├── fit_model.json                 # bonus model coefficients (~2KB text)
│   └── job_requisitions.json           # sample requisitions used during development
├── docs/
│   ├── architecture_diagram.png       # overall system sketch (Figure 1)
│   ├── langgraph_visualization.png    # graph visualization (Figure 2, mandatory)
│   ├── system_specification.pdf        # this document
│   └── demo_script.md                 # 7 usage scenarios
├── n8n/
│   ├── build_v2_workflow.py            # generates the workflow JSON (never hand-edited)
│   └── hr-copilot-v2-workflow.json
├── n8n_data/                          # calendar.json / sent_emails.json / ats_log.json
├── training/                          # OFFLINE only, not copied into the image – see Section 9
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

Quick start

```
cp .env.example .env                # paste OPENAI_API_KEY, QDRANT_URL, QDRANT_API_KEY
docker compose up --build -d        # brings up n8n, the agent and the UI

curl "http://localhost:5001/health?deep=true"    # confirm before demoing

curl -X POST http://localhost:5679/webhook/screen \
  -H "Content-Type: application/json" \
  -d '{"requisition_text": "Senior Backend Engineer, 5+ years Python and AWS."}'

# or open the UI:
http://localhost:8502
```

9 Bonus (Optional) — Statistical Fit Model as a Tool

This section documents spec §4.6's optional bonus. It is a genuinely separate, self-contained piece of the project (its own tool, its own offline training pipeline, its own coefficients file), kept out of the core requirements above on purpose: every claim in Sections 1–8 is true with or without it — the Evaluator (Node 3) degrades gracefully to CV-only reasoning if the tool is removed.

What it is

A classic machine-learning model — **ridge regression** — fitted offline on 294 hand-labelled (requisition, candidate) pairs across 21 candidates, used as a second, independent opinion the autonomous Evaluator agent can call as a tool (`statistical_fit_score` , Section 5) before committing to its own score.

Features (5)

Feature	What it captures
Keyword skill overlap	Fraction of required skills found by keyword match in the CV.
Years above minimum	How far the candidate's experience exceeds the requisition's floor.
Years shortfall	How far short, when below the floor (rare in production — retrieval already filters most of these out).
Job-title overlap	Textual overlap between the candidate's current title and the role title.
Vector-search similarity	The Qdrant similarity score already computed by Node 2 — previously computed on every run and used by nothing.

Served without scikit-learn

Trained offline (throwaway container, scikit-learn only there); the runtime loads `data/fit_model.json` (coefficients, ~2 KB of text) and evaluates it with a plain dot product in pure Python (`src/scoring_model.py`). scikit-learn is never installed in the image that serves the API, the UI, or ingestion — keeping ~120 MB out of the image and making the "model" a diffable text file rather than a pickle. The trainer verifies the pure-Python path reproduces scikit-learn's own output across every training row, and refuses to write the artifact at all if the two disagree by more than 1×10^{-6} .

Honest performance

	leave-one-candidate-out	leave-one-requisition-out
MAE	0.77 points	0.77 points
Band accuracy	89%	90%

Both splits are grouped (never random — the same CV appears in 14 rows), because a random split would leak a candidate across folds.

- **It barely beats the best single feature.** Keyword skill overlap alone scores 0.84 MAE against the model's 0.77 — a margin well inside the fold-to-fold spread, so the honest claim is that five features are no worse than the single best one, and what they add is an explainable coefficient table, not a large accuracy win.
- **It recovers 50% of strong candidates** (misses 13 of 26) — the number that actually matters for a screening aid, since 83% of pairs are obvious no-fits and overall accuracy would flatter the model. The misses are concentrated

in the genuinely ambiguous pairs, where the candidate is a defensible fit on job family but thin on literal keyword evidence; a larger and more balanced training set is what this number needs, not a different model class. It is also why the estimate stays advisory and never touches the pass/fail threshold.

- **Labels were reviewed with steering** toward the ~29 ambiguous rows; no model prediction was shown during labelling (so this is not leakage), but it is not a blind sample either.
- **It declines rather than guessing when it has no skills signal.** Keyword skill overlap carries the largest coefficient, so a requisition that names no skills at all ("software engineer") leaves the remaining four features producing roughly the same low score for every candidate — strong ones included. In that case the tool returns no estimate and says why, which both stops it anchoring the Evaluator downward and stops Node 6 reporting a disagreement against a comparison that was never meaningfully made.

Design decisions worth defending in the presentation

- **A per-candidate closure, not a function taking arguments** — an LLM cannot supply a real cosine similarity, so letting it fill in the features would mean letting it invent them, destroying the independence that makes disagreement meaningful.
- **A range, not a point estimate** — a bare 8.4 was measured to make gpt-4o-mini echo 8.4 as its own score and restate the tool's reasoning as its justification. Leading with a range and a list of what the model could not see broke that anchoring.
- **Disagreement is the payoff, not accuracy.** Because the two are trained/reasoned independently, a ≥ 2.5 -point gap between them (surfaced in Node 6's report) is a real signal — usually the LLM reading depth the keyword features miss, occasionally the LLM talking itself into a candidate the numbers do not support.

Known limitation shared by both halves

Requirements phrased in business language rather than technology names ("full-cycle pipeline management", "quota attainment") appear verbatim in no CV, so the keyword feature collapses — and the LLM Evaluator degrades the same way for the same reason. Because both fail in the same direction, the disagreement flag cannot catch this particular failure mode. This is a property of the whole design, not just the statistical half.